Employee Leave Request Workflow

Week 8 Practice - Question 2

Design: Employee has many LeaveRequests. FullTimeEmployee, PartTimeEmployee, and Contractor specialize Employee.
LeaveRequest privately manages Pending, Approved, Rejected and blocks a return to Pending after review.
Java solution:

```
import java.time.*;
abstract class Employee {final String name;Employee(String n){name=n;}abstract boolean
canRequestLeave(int days);}
class FullTimeEmployee extends Employee {FullTimeEmployee(String n){super(n);}boolean
canRequestLeave(int d){return d>0;}}
class PartTimeEmployee extends Employee {PartTimeEmployee(String n){super(n);}boolean
canRequestLeave(int d){return d>0&&d<=10;}}
class Contractor extends Employee {Contractor(String n){super(n);}boolean canRequestLeave(int
d){return d>0&&d<=5;}}
class LeaveRequest {enum Status{Pending,Approved,Rejected}final Employee employee;final
LocalDate start,end;private Status status=Status.Pending;
LeaveRequest(Employee e,LocalDate s,LocalDate en){if(en.isBefore(s))throw new
IllegalArgumentException("Invalid dates");employee=e;start=s;end=en;}Status status(){return
status;}
void review(boolean approve){if(status!=Status.Pending)throw new
IllegalStateException("Request already
reviewed");status=approve?Status.Approved:Status.Rejected;}
void setStatus(Status s){if(status!=Status.Pending||s==Status.Pending)throw new
IllegalStateException("Cannot change leave request status from "+status+" to "+s);status=s;}
}
public class Main {public static void main(String[] args){Employee j=new
FullTimeEmployee("John"),n=new PartTimeEmployee("Jane");
LeaveRequest a=new
LeaveRequest(j,LocalDate.of(2026,1,1),LocalDate.of(2026,1,5));System.out.println("Leave
request submitted for John (Jan 1-5). Status:
"+a.status()+".");a.review(true);System.out.println("John’s leave request (Jan 1-5) approved.
Status: "+a.status()+".");
LeaveRequest b=new
LeaveRequest(n,LocalDate.of(2026,2,10),LocalDate.of(2026,2,11));System.out.println("Leave
request submitted for Jane (Feb 10-11). Status:
"+b.status()+".");b.review(false);System.out.println("Jane’s leave request (Feb 10-11)
rejected. Status: "+b.status()+".");
try{a.setStatus(LeaveRequest.Status.Pending);}catch(IllegalStateException
e){System.out.println(e.getMessage());}}}
```

Expected: John is Approved; Jane is Rejected; an Approved request cannot revert to Pending.
Numeric employee policy limits are illustrative because the source does not define exact policy amounts.